ZeroSum research memo

Question. In a two-player zero-sum game with hidden information, an equilibrium strategy cannot lose on average. Against a flawed opponent it also does not win as much as it could. How fast can the equilibrium be computed, how much does it leave behind, and what does taking more cost?

Method. Kuhn poker (12 information sets, solved on paper in 1950) and Leduc hold'em (288 information sets, the standard small research game) are solved five ways by regret minimization and once, exactly, by a linear program that shares no code with the solvers. All amounts are in milli-antes per hand (mA): one thousandth of the forced bet. Everything is exact game-tree arithmetic except the sampled solver, which is seeded.

1. The solvers agree with an exact method

"Exploitability" is what a perfect opponent wins per hand against a strategy; it is zero only at equilibrium, and it brackets the game's true value.

Game	Information sets	LP value	LP strategy's exploitability	CFR+ bracket on the value	LP value inside
kuhn	12	-0.055555556	2.8e-14 mA	[-0.0555571, -0.0555528]	yes
leduc	288	-0.085606424	7.6e-13 mA	[-0.0856111, -0.0856016]	yes
leduc5	780	-0.112768934	4.0e-13 mA	[-0.1127873, -0.1127539]	yes
leduc13	5,148	-0.092673901	5.2e-13 mA	[-0.0930105, -0.0922828]	yes

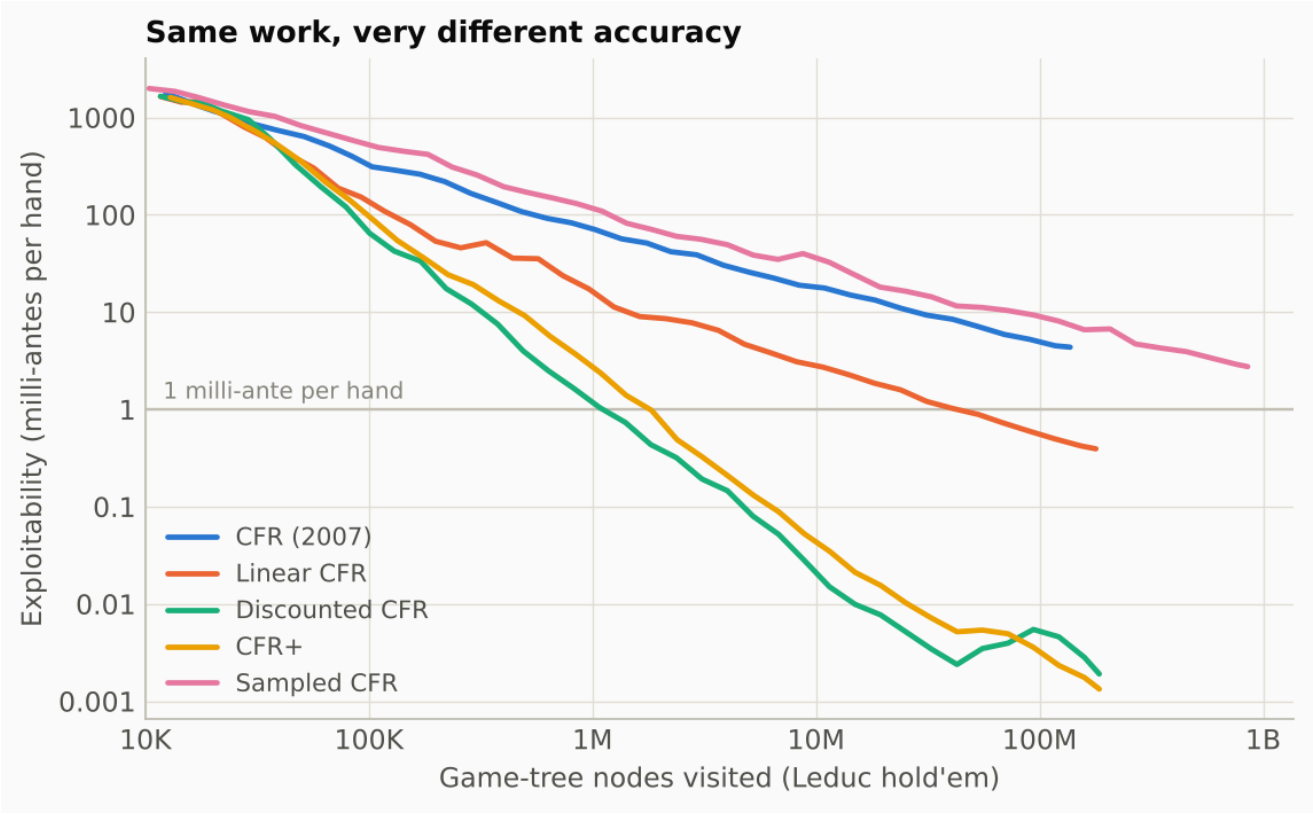
- Kuhn poker's value is -0.055555556 = -1/18, as Kuhn derived.
- Leduc's value is -0.085606424 antes per hand for the first player.
- The linear program's strategy, evaluated by the C++ best-response code, is exploitable for less than a trillionth of an ante. Two unrelated methods and two separate evaluators agree.
- The rules are also written twice (C++ deals ranks, a Python reference deals cards from a deck). Both give the same information sets and the same value. Without merging suits Leduc has 936 information sets; merged, 288.

2. Five algorithms, one target

Each algorithm ran on Leduc until a perfect opponent could win no more than 1 mA per hand, with the same budget of game-tree node visits.

Algorithm	Reached 1 mA	Iterations	Nodes visited	Seconds	Final mA	Less work than CFR
CFR (2007)	yes	842,438	2,368,783,718	15.707	0.95	1×
Linear CFR	yes	11,515	40,632,503	0.298	0.99	58×
Discounted CFR	yes	321	1,175,470	0.009	0.94	2,015×
CFR+	yes	470	1,737,984	0.013	0.90	1,363×
Sampled CFR	no	93,607,453	4,000,000,021	42.687	1.36	not reached

- **Discounted CFR needed 2,015 times fewer node visits than the original CFR** to reach the same accuracy (321 iterations against 842,438). Node visits are counted, not timed, so the ratio does not depend on how fast the machine is. The exact stopping iteration is sensitive to the last bit of floating-point arithmetic (accuracy is checked every 5% of progress and does not fall smoothly), so read these as "about two thousand", not as constants. The build turns off fused multiply-add so that GCC and Clang, and x86 and ARM, perform the same arithmetic.
- Full traversals ran at about 135 million node visits per second on one core of the test machine (Linux-6.18.44-fc-v80-x86_64-with-glibc2.39).
- Sampling did not pay here. External-sampling CFR touches a sliver of the tree per iteration, but Leduc's whole tree is 1,936 nodes: a full pass is already cheap, and the sampled regrets are noisy. Its case is games too large to walk, which this project does not include.



Kuhn's equilibria

Kuhn showed the first player has a one-parameter family of equilibria: bluff the Jack with any probability a up to 1/3, bet the King 3a, and call a bet with the Queen a + 1/3. The second player's equilibrium is unique: bluff the Jack 1/3 and call with the Queen 1/3.

Solver	a = bluff J	bet K	bet K / a	call with Q	call Q − a	2nd: bluff J	2nd: call Q
CFR (2007)	0.205	0.624	3.04	0.541	0.336	0.336	0.333
Linear CFR	0.218	0.655	3.00	0.552	0.334	0.333	0.333
Discounted CFR	0.187	0.562	3.01	0.521	0.334	0.333	0.333
CFR+	0.233	0.699	3.00	0.566	0.333	0.333	0.333
Sampled CFR	0.324	0.972	3.00	0.660	0.336	0.332	0.336
Linear program	0.000	0.000	n/a	0.333	0.333	0.333	0.333

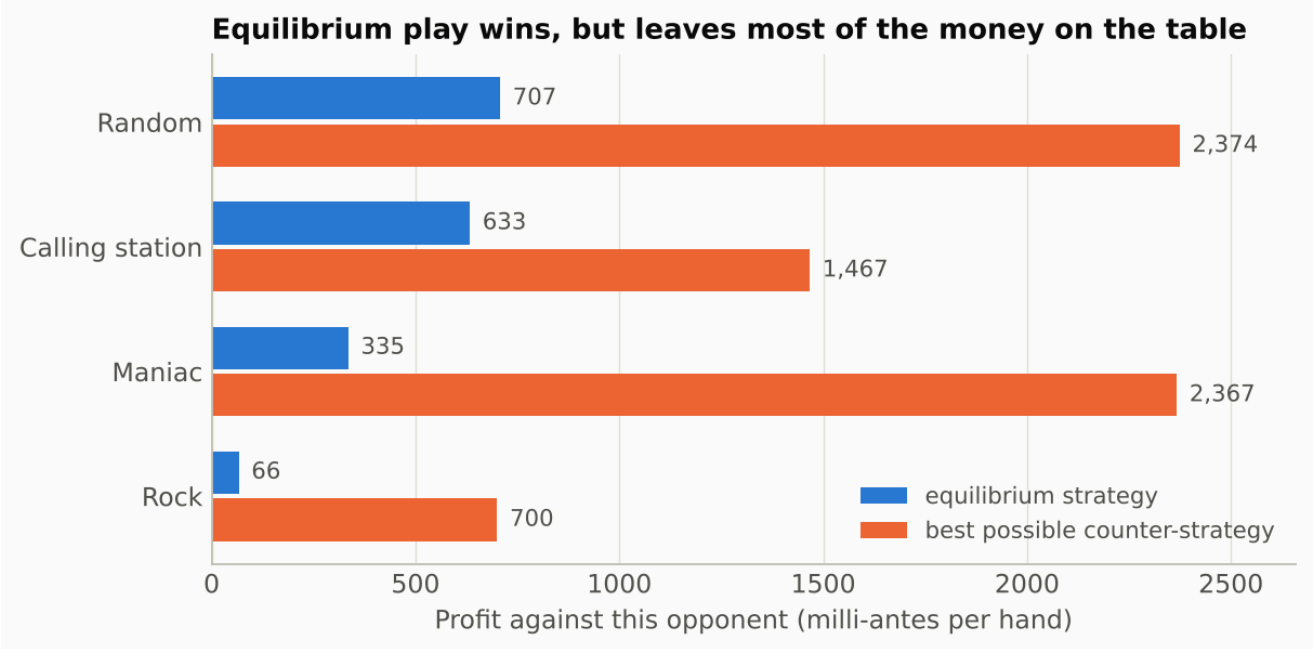
Different solvers land on different members of the family, and all satisfy Kuhn's three relations. The linear program returns a corner (a = 0): a first player who never bets at all, which is still unexploitable.

3. Equilibrium against opponents who make mistakes

Four simple opponent types play Leduc. "Maximum" is the best counter-strategy to that exact opponent.

Opponent	How it plays	Equilibrium (CFR+)	Equilibrium (LP)	Maximum	CFR+ share	Risk of the maximum
Random	picks every action equally often	707.1	667.2	2,373.6	30%	3,417
Calling station	never folds, never bets	632.9	607.0	1,466.7	43%	1,352
Maniac	bets and raises at every chance	334.7	240.3	2,366.7	14%	1,085
Rock	only puts money in with a King or a pair	66.1	32.8	700.0	9%	963

- The equilibrium strategy profits from every one of them without knowing anything about them, but it collects between 9% and 43% of what is available.
- **Equilibria are not interchangeable against imperfect opponents.** Both equilibrium columns are unexploitable, yet against the Rock one earns 66.1 mA and the other 32.8 mA. Which equilibrium a solver lands on is an accident of the algorithm, and it matters.
- The maximum is dangerous. The counter-strategy that takes everything from the Rock can itself be beaten for 963 mA per hand if the read is wrong, more than it was winning.



4. Taking more without betting everything on the read

Two ways to lean toward the exploit:

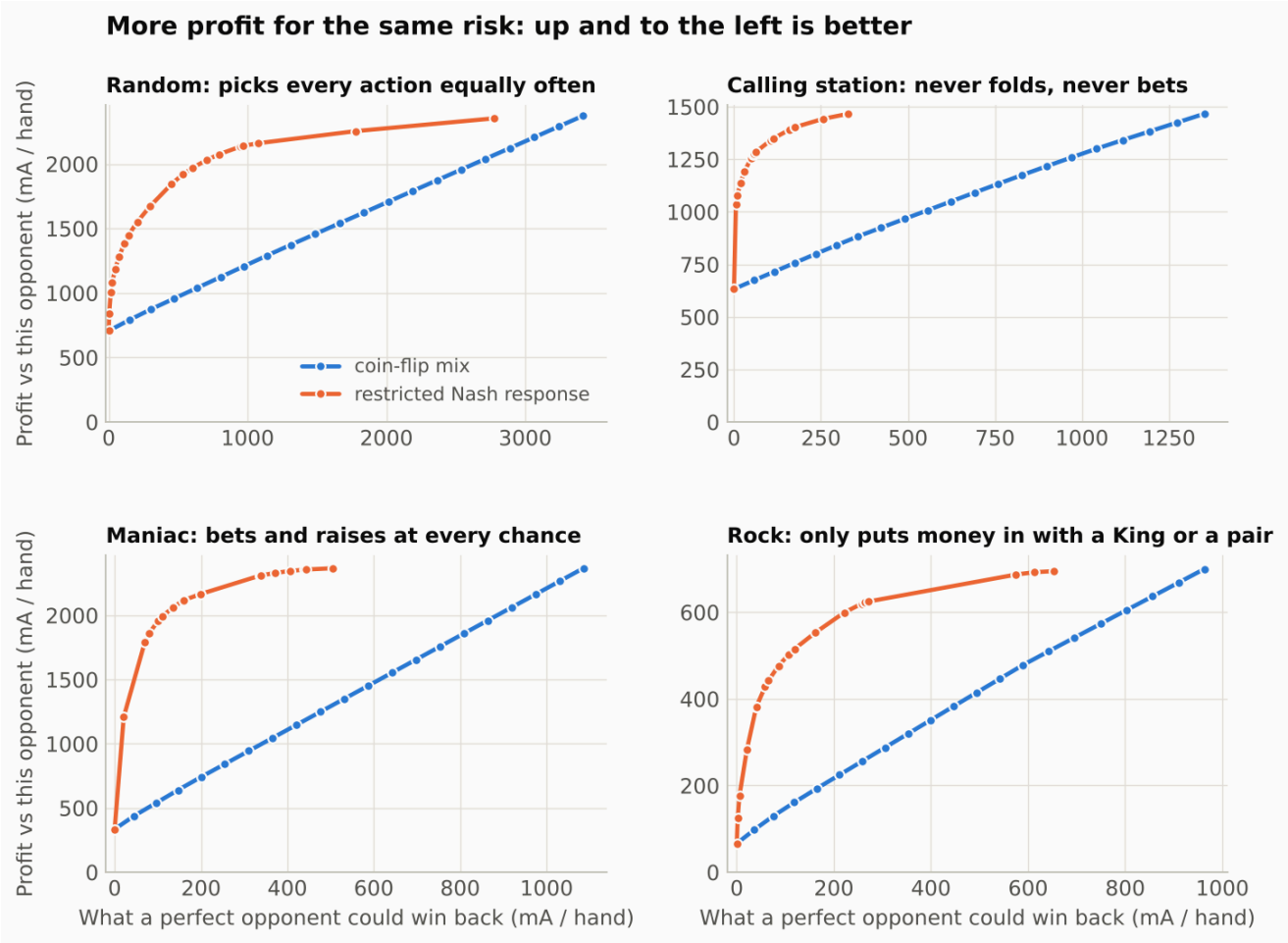
- **Coin-flip mix.** Before each hand, play the counter-strategy with probability p and the equilibrium otherwise.

- **Restricted Nash response** (Johanson, Zinkevich and Bowling, 2007). Solve a modified game in which the opponent is the modelled type with probability p and a perfect adversary otherwise. Its solution is the most profitable strategy among all strategies with at most that much exposure.

For each opponent, the risk each method takes on to capture half of the profit that lies beyond equilibrium play (interpolated along each curve):

Opponent	Profit target (mA)	Risk, coin-flip mix	Risk, restricted response	Risk ratio
Random	1,540	1,663	199	8.4×
Calling station	1,050	624	7	86.6×
Maniac	1,351	532	33	16.1×
Rock	383	447	41	10.9×

Against the Rock with p = 0.2, the restricted response earns 381 mA per hand (54% of the maximum) while a perfect opponent could win back only 40 mA. The coin-flip mix needs 444 mA of exposure for the same profit: 11 times as much.



The reason is where the risk is taken. The mix plays the whole counter-strategy some of the time, including the parts that give up a lot for little. The restricted response changes only the decisions where the model's mistake is large relative to the exposure, and keeps equilibrium play elsewhere.

5. Larger decks

The same rules with more ranks, 1,000 iterations of Discounted CFR each.

Ranks	Nodes	Information sets	Seconds	Exploitability (mA)	Value bracket
3	1,936	288	0.03	0.176	[-0.08577, -0.08542]
4	4,741	504	0.06	0.114	[-0.10234, -0.10211]
5	9,376	780	0.12	0.092	[-0.11285, -0.11267]
6	16,291	1,116	0.19	0.077	[-0.10462, -0.10447]
7	25,936	1,512	0.33	0.072	[-0.10239, -0.10225]
8	38,761	1,968	0.44	0.065	[-0.09917, -0.09905]
9	55,216	2,484	0.68	0.106	[-0.10013, -0.09991]
10	75,751	3,060	0.88	0.107	[-0.09868, -0.09847]
11	100,816	3,696	1.08	0.059	[-0.09575, -0.09563]
12	130,861	4,392	1.39	0.147	[-0.09316, -0.09287]
13	166,336	5,148	1.64	0.081	[-0.09274, -0.09258]

Limits

- These are toy games. Leduc has 288 information sets; no-limit hold'em has about 10^161. Nothing here is a poker bot.
- The opponent types are fixed and known exactly. A real opponent has to be estimated from a finite number of hands, and adapts.
- Trees are stored explicitly, so memory grows with the game. The methods that scale further (abstraction, depth-limited search, learned value functions) are not implemented.
- Timings are from one shared cloud machine and vary run to run. Node counts, iteration counts, values, and exploitabilities do not.

Reproduce

```
make
python3 scripts/build_report.py build
```

Ran in 224 s with Python 3.13.16, NumPy 2.5.3, SciPy 1.18.1.

References

- Kuhn, H. (1950). A simplified two-person poker. *Contributions to the Theory of Games* 1.
- Koller, D., Megiddo, N., von Stengel, B. (1994). Fast algorithms for finding randomized strategies in game trees. *STOC*.
- Southey, F. et al. (2005). Bayes' bluff: opponent modelling in poker. *UAI*. (Leduc hold'em.)
- Zinkevich, M., Johanson, M., Bowling, M., Piccione, C. (2007). Regret minimization in games with incomplete information. *NeurIPS*.
- Johanson, M., Zinkevich, M., Bowling, M. (2007). Computing robust counter-strategies. *NeurIPS*.
- Lanctot, M., Waugh, K., Zinkevich, M., Bowling, M. (2009). Monte Carlo sampling for regret minimization in extensive games. *NeurIPS*.
- Tammelin, O. (2014). Solving large imperfect information games using CFR+. *arXiv:1407.5042*.

- Brown, N., Sandholm, T. (2019). Solving imperfect-information games via discounted regret minimization. *AAAI*.